

Fashion-MNIST Classification Project

This project aims to build and compare different deep learning models for classifying images from the Fashion-MNIST dataset. We will explore the impact of increasing model complexity, from a simple Artificial Neural Network (ANN) to more sophisticated Convolutional Neural Networks (CNNs), on performance and efficiency.

Project Agenda & Steps:

1. **Dataset Setup:**
 - Import necessary libraries.
 - Load the Fashion-MNIST dataset.
 - Preprocess the data (normalize and reshape images, one-hot encode labels).
 - Verify the shapes of the processed data.
2. **Model Building:**
 - Define the architecture for each model:
 - Basic ANN Model
 - Basic CNN Model
 - Deeper CNN Model
3. **Model Training:**
 - Train each model using the prepared training data.
 - Implement Early Stopping and Model Checkpointing to optimize training and save the best model weights.
4. **Model Evaluation:**
 - Load the best weights for each trained model.
 - Evaluate each model's performance on the test set using metrics like loss and accuracy.
 - Visualize the training history (accuracy and loss curves) for comparison.
 - Generate and visualize confusion matrices to understand model performance on each class.
5. **Prediction Analysis:**
 - Use the best performing model (Basic CNN in this case) to make predictions on the test set.
 - Identify and visualize examples of both correctly and incorrectly classified images to gain insights into model behavior.

Goal: To analyze and demonstrate how model complexity influences classification accuracy and efficiency on the Fashion-MNIST dataset.

1. Dataset Setup Documentation

1.1) Library Imports

This section imports the essential libraries:

- **NumPy** for numerical operations
- **TensorFlow / Keras** for building and training deep learning models
- **Matplotlib** for visualizations

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

1.2) Load Dataset

Load the **Fashion-MNIST** dataset from Keras, splitting it into:

- `train_images, train_labels` → training set
- `test_images, test_labels` → test set

```
(train_images, train_labels), (test_images, test_labels) =
keras.datasets.fashion_mnist.load_data()
```

1.3) Normalize Data

Scale pixel values from **0–255** to the range **0–1** for faster and more stable training.

Example:

- Before: a pixel value could be 200
- After: $200 / 255 \approx 0.78$

```
train_images = train_images / 255.0
test_images = test_images / 255.0
```

1.4) Reshape Data

Reshape images to **28×28×1** by adding a channel dimension, making them compatible with CNN layers.

Example:

- Before: `(60000, 28, 28)` → 60,000 grayscale images without channel info
- After: `(60000, 28, 28, 1)` → channel dimension (1) added for CNN input

```
train_images = train_images.reshape((train_images.shape[0], 28, 28, 1))
test_images = test_images.reshape((test_images.shape[0], 28, 28, 1))
```

1.5) One-Hot Encode Labels

Convert class labels (0–9) into **one-hot encoded vectors** with 10 output classes, required for multi-class classification.

Example:

- Original label: 3
- One-hot encoded: [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]

```
train_labels_one_hot = keras.utils.to_categorical(train_labels, num_classes=10)
test_labels_one_hot = keras.utils.to_categorical(test_labels, num_classes=10)
```

1.6) Verify Dataset Shapes

Print the shapes of images and labels after preprocessing to confirm everything is in the expected format.

Example output:

- Training images shape: (60000, 28, 28, 1)
- Testing images shape: (10000, 28, 28, 1)
- Training labels shape: (60000, 10)
- Testing labels shape: (10000, 10)

```
print("Training images shape:", train_images.shape)
print("Testing images shape:", test_images.shape)
print("Training labels shape:", train_labels_one_hot.shape)
print("Testing labels shape:", test_labels_one_hot.shape)

Training images shape: (60000, 28, 28, 1)
Testing images shape: (10000, 28, 28, 1)
Training labels shape: (60000, 10)
Testing labels shape: (10000, 10)
```

2. Model Building

2.1) Basic ANN Model

- **Flatten (28×28→784)** → 0 params

- **Dense (128, ReLU)** → 100,480 params
- **Dense (64, ReLU)** → 8,256 params
- **Dense (10, Softmax)** → 650 params

Total params: ~109K

Choosing the Right Activation Function

The choice of activation function depends on the type of problem you are trying to solve. Here are some guidelines:

- **For binary classification:** Use the sigmoid activation function in the output layer. It will squash outputs between 0 and 1, representing probabilities for the two classes.
- **For multi-class classification:** Use the softmax activation function in the output layer. It will output probability distributions over all classes.
- **If unsure:** Use the ReLU activation function in the hidden layers. ReLU is the most common default activation function and usually a good choice.

```
ann_model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28, 1)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

ann_model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])
```

```
ann_model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/
flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

```
Model: "sequential"
```

Layer (type)	Output Shape
Param #	
flatten (Flatten)	(None, 784)


```
metrics=['accuracy'])
```

```
basic_cnn_model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Model: "sequential_1"
```

Layer (type) Param #	Output Shape	
conv2d (Conv2D) 320	(None, 26, 26, 32)	
max_pooling2d (MaxPooling2D) 0	(None, 13, 13, 32)	
conv2d_1 (Conv2D) 18,496	(None, 11, 11, 64)	
max_pooling2d_1 (MaxPooling2D) 0	(None, 5, 5, 64)	
flatten_1 (Flatten) 0	(None, 1600)	
dense_3 (Dense) 102,464	(None, 64)	
dense_4 (Dense) 650	(None, 10)	

```
Total params: 121,930 (476.29 KB)
```

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

2.3) Deeper CNN Model

- **Conv2D (32, 3×3) + BN + MaxPool + Dropout** → 320 + 64 params
- **Conv2D (64, 3×3) + BN + MaxPool + Dropout** → 18,496 + 128 params
- **Conv2D (128, 3×3) + BN + MaxPool + Dropout** → 73,856 + 256 params
- **Flatten**
- **Dense (128, ReLU) + BN + Dropout** → 147,584 + 256 params
- **Dense (10, Softmax)** → 1,290 params

Total params: ~242K

```
deeper_cnn_model = keras.Sequential([
    keras.layers.Conv2D(32, (3, 3), activation='relu',
input_shape=(28, 28, 1)),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Dropout(0.25),

    keras.layers.Conv2D(64, (3, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Dropout(0.25),

    keras.layers.Conv2D(128, (3, 3), activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPooling2D((2, 2)),
    keras.layers.Dropout(0.25),

    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.BatchNormalization(),
    keras.layers.Dropout(0.5),

    keras.layers.Dense(10, activation='softmax')
])

deeper_cnn_model.compile(optimizer='adam',
                        loss='categorical_crossentropy',
                        metrics=['accuracy'])

deeper_cnn_model.summary()
```

Model: "sequential_2"

Layer (type) Param #	Output Shape	
conv2d_2 (Conv2D) 320	(None, 26, 26, 32)	
batch_normalization 128 (BatchNormalization)	(None, 26, 26, 32)	
max_pooling2d_2 (MaxPooling2D) 0	(None, 13, 13, 32)	
dropout (Dropout) 0	(None, 13, 13, 32)	
conv2d_3 (Conv2D) 18,496	(None, 11, 11, 64)	
batch_normalization_1 256 (BatchNormalization)	(None, 11, 11, 64)	
max_pooling2d_3 (MaxPooling2D) 0	(None, 5, 5, 64)	
dropout_1 (Dropout) 0	(None, 5, 5, 64)	
conv2d_4 (Conv2D) 73,856	(None, 3, 3, 128)	
batch_normalization_2 512	(None, 3, 3, 128)	

	(BatchNormalization)		
0	max_pooling2d_4 (MaxPooling2D)	(None, 1, 1, 128)	
0	dropout_2 (Dropout)	(None, 1, 1, 128)	
0	flatten_2 (Flatten)	(None, 128)	
16,512	dense_5 (Dense)	(None, 128)	
512	batch_normalization_3	(None, 128)	
	(BatchNormalization)		
0	dropout_3 (Dropout)	(None, 128)	
1,290	dense_6 (Dense)	(None, 10)	
Total params: 111,882 (437.04 KB)			
Trainable params: 111,178 (434.29 KB)			
Non-trainable params: 704 (2.75 KB)			

3. Train models

3.1) Training ANN Model

- **EarlyStopping:** monitors validation loss, stops if no improvement for 5 epochs, restores best weights.

- **ModelCheckpoint:** saves best ANN weights (`best_ann_model_weights.weights.h5`) based on validation loss.
- **Training:**
 - Epochs: up to 30 (early stopping controls actual run)
 - Batch size: 64
 - Validation on test set
 - Callbacks: EarlyStopping + ModelCheckpoint

```
early_stopping = keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

model_checkpoint_ann = keras.callbacks.ModelCheckpoint(
    filepath='best_ann_model_weights.weights.h5',
    monitor='val_loss',
    save_best_only=True,
    save_weights_only=True,
    mode='min',
    verbose=1
)

print("Training ANN model...")
ann_history = ann_model.fit(
    train_images,
    train_labels_one_hot,
    epochs=30,
    batch_size=60,
    validation_data=(test_images, test_labels_one_hot),
    callbacks=[early_stopping, model_checkpoint_ann]
)
print("ANN model training finished.")
```

Training ANN model...

Epoch 1/30
 987/1000 ————— 0s 2ms/step - accuracy: 0.7728 - loss: 0.6633
 Epoch 1: val_loss improved from inf to 0.42304, saving model to best_ann_model_weights.weights.h5
 1000/1000 ————— 6s 4ms/step - accuracy: 0.7735 - loss: 0.6611 - val_accuracy: 0.8498 - val_loss: 0.4230

Epoch 2/30
 986/1000 ————— 0s 2ms/step - accuracy: 0.8654 - loss: 0.3733
 Epoch 2: val_loss did not improve from 0.42304

```
1000/1000 _____ 3s 3ms/step - accuracy: 0.8654 - loss:
0.3732 - val_accuracy: 0.8453 - val_loss: 0.4239
Epoch 3/30
 992/1000 _____ 0s 3ms/step - accuracy: 0.8778 - loss:
0.3345
Epoch 3: val_loss improved from 0.42304 to 0.37379, saving model to
best_ann_model_weights.weights.h5
1000/1000 _____ 4s 4ms/step - accuracy: 0.8778 - loss:
0.3345 - val_accuracy: 0.8661 - val_loss: 0.3738
Epoch 4/30
 995/1000 _____ 0s 2ms/step - accuracy: 0.8867 - loss:
0.3112
Epoch 4: val_loss improved from 0.37379 to 0.35612, saving model to
best_ann_model_weights.weights.h5
1000/1000 _____ 3s 3ms/step - accuracy: 0.8867 - loss:
0.3112 - val_accuracy: 0.8713 - val_loss: 0.3561
Epoch 5/30
 984/1000 _____ 0s 2ms/step - accuracy: 0.8937 - loss:
0.2905
Epoch 5: val_loss improved from 0.35612 to 0.34403, saving model to
best_ann_model_weights.weights.h5
1000/1000 _____ 3s 3ms/step - accuracy: 0.8937 - loss:
0.2906 - val_accuracy: 0.8761 - val_loss: 0.3440
Epoch 6/30
 995/1000 _____ 0s 2ms/step - accuracy: 0.8969 - loss:
0.2795
Epoch 6: val_loss improved from 0.34403 to 0.34185, saving model to
best_ann_model_weights.weights.h5
1000/1000 _____ 3s 3ms/step - accuracy: 0.8969 - loss:
0.2795 - val_accuracy: 0.8790 - val_loss: 0.3419
Epoch 7/30
 986/1000 _____ 0s 3ms/step - accuracy: 0.8997 - loss:
0.2707
Epoch 7: val_loss improved from 0.34185 to 0.33053, saving model to
best_ann_model_weights.weights.h5
1000/1000 _____ 4s 4ms/step - accuracy: 0.8997 - loss:
0.2707 - val_accuracy: 0.8822 - val_loss: 0.3305
Epoch 8/30
 990/1000 _____ 0s 2ms/step - accuracy: 0.9059 - loss:
0.2513
Epoch 8: val_loss did not improve from 0.33053
1000/1000 _____ 3s 3ms/step - accuracy: 0.9059 - loss:
0.2514 - val_accuracy: 0.8762 - val_loss: 0.3451
Epoch 9/30
 996/1000 _____ 0s 2ms/step - accuracy: 0.9053 - loss:
0.2507
Epoch 9: val_loss improved from 0.33053 to 0.32551, saving model to
best_ann_model_weights.weights.h5
1000/1000 _____ 3s 3ms/step - accuracy: 0.9053 - loss:
```

```
0.2506 - val_accuracy: 0.8815 - val_loss: 0.3255
Epoch 10/30
 999/1000 _____ 0s 2ms/step - accuracy: 0.9075 - loss:
0.2433
Epoch 10: val_loss did not improve from 0.32551
1000/1000 _____ 3s 3ms/step - accuracy: 0.9075 - loss:
0.2433 - val_accuracy: 0.8846 - val_loss: 0.3391
Epoch 11/30
1000/1000 _____ 0s 3ms/step - accuracy: 0.9138 - loss:
0.2264
Epoch 11: val_loss did not improve from 0.32551
1000/1000 _____ 3s 3ms/step - accuracy: 0.9138 - loss:
0.2264 - val_accuracy: 0.8826 - val_loss: 0.3338
Epoch 12/30
 990/1000 _____ 0s 3ms/step - accuracy: 0.9160 - loss:
0.2227
Epoch 12: val_loss did not improve from 0.32551
1000/1000 _____ 3s 3ms/step - accuracy: 0.9160 - loss:
0.2227 - val_accuracy: 0.8814 - val_loss: 0.3451
Epoch 13/30
 989/1000 _____ 0s 2ms/step - accuracy: 0.9190 - loss:
0.2140
Epoch 13: val_loss improved from 0.32551 to 0.32431, saving model to
best_ann_model_weights.weights.h5
1000/1000 _____ 3s 3ms/step - accuracy: 0.9190 - loss:
0.2140 - val_accuracy: 0.8882 - val_loss: 0.3243
Epoch 14/30
 982/1000 _____ 0s 2ms/step - accuracy: 0.9224 - loss:
0.2022
Epoch 14: val_loss did not improve from 0.32431
1000/1000 _____ 3s 3ms/step - accuracy: 0.9224 - loss:
0.2023 - val_accuracy: 0.8846 - val_loss: 0.3489
Epoch 15/30
 995/1000 _____ 0s 4ms/step - accuracy: 0.9227 - loss:
0.2020
Epoch 15: val_loss did not improve from 0.32431
1000/1000 _____ 4s 4ms/step - accuracy: 0.9227 - loss:
0.2020 - val_accuracy: 0.8806 - val_loss: 0.3455
Epoch 16/30
 982/1000 _____ 0s 3ms/step - accuracy: 0.9279 - loss:
0.1921
Epoch 16: val_loss did not improve from 0.32431
1000/1000 _____ 3s 3ms/step - accuracy: 0.9279 - loss:
0.1922 - val_accuracy: 0.8789 - val_loss: 0.3575
Epoch 17/30
 983/1000 _____ 0s 2ms/step - accuracy: 0.9287 - loss:
0.1896
Epoch 17: val_loss did not improve from 0.32431
1000/1000 _____ 3s 3ms/step - accuracy: 0.9287 - loss:
```

```

0.1896 - val_accuracy: 0.8859 - val_loss: 0.3474
Epoch 18/30
 986/1000 _____ 0s 2ms/step - accuracy: 0.9315 - loss:
0.1795
Epoch 18: val_loss did not improve from 0.32431
1000/1000 _____ 3s 3ms/step - accuracy: 0.9315 - loss:
0.1796 - val_accuracy: 0.8883 - val_loss: 0.3530
ANN model training finished.

```

3.2) Training Basic CNN Model

- **ModelCheckpoint:** saves best Basic CNN weights (best_basic_cnn_model_weights.weights.h5) based on validation loss.
- **Training:**
 - Epochs: up to 30 (early stopping controls actual run)
 - Batch size: 64
 - Validation on test set
 - Callbacks: EarlyStopping + ModelCheckpoint

```

model_checkpoint_basic_cnn = keras.callbacks.ModelCheckpoint(
    filepath='best_basic_cnn_model_weights.weights.h5',
    monitor='val_loss',
    save_best_only=True,
    save_weights_only=True,
    mode='min',
    verbose=1
)

print("Training Basic CNN model...")
basic_cnn_history = basic_cnn_model.fit(
    train_images,
    train_labels_one_hot,
    epochs=30,
    batch_size=64,
    validation_data=(test_images, test_labels_one_hot),
    callbacks=[early_stopping, model_checkpoint_basic_cnn]
)
print("Basic CNN model training finished.")

```

Training Basic CNN model...

```

Epoch 1/30
938/938 _____ 0s 4ms/step - accuracy: 0.7449 - loss:
0.7295
Epoch 1: val_loss improved from inf to 0.38625, saving model to
best_basic_cnn_model_weights.weights.h5

```

```
938/938 _____ 10s 6ms/step - accuracy: 0.7450 - loss:
0.7292 - val_accuracy: 0.8633 - val_loss: 0.3863
Epoch 2/30
935/938 _____ 0s 3ms/step - accuracy: 0.8748 - loss:
0.3489
Epoch 2: val_loss improved from 0.38625 to 0.34199, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 _____ 3s 3ms/step - accuracy: 0.8748 - loss:
0.3489 - val_accuracy: 0.8761 - val_loss: 0.3420
Epoch 3/30
927/938 _____ 0s 4ms/step - accuracy: 0.8900 - loss:
0.2999
Epoch 3: val_loss improved from 0.34199 to 0.30557, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.8900 - loss:
0.2999 - val_accuracy: 0.8906 - val_loss: 0.3056
Epoch 4/30
933/938 _____ 0s 3ms/step - accuracy: 0.9008 - loss:
0.2693
Epoch 4: val_loss improved from 0.30557 to 0.28382, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.9008 - loss:
0.2693 - val_accuracy: 0.8969 - val_loss: 0.2838
Epoch 5/30
928/938 _____ 0s 3ms/step - accuracy: 0.9128 - loss:
0.2404
Epoch 5: val_loss improved from 0.28382 to 0.27290, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 _____ 3s 4ms/step - accuracy: 0.9128 - loss:
0.2404 - val_accuracy: 0.8987 - val_loss: 0.2729
Epoch 6/30
934/938 _____ 0s 3ms/step - accuracy: 0.9164 - loss:
0.2245
Epoch 6: val_loss did not improve from 0.27290
938/938 _____ 4s 4ms/step - accuracy: 0.9164 - loss:
0.2245 - val_accuracy: 0.8993 - val_loss: 0.2737
Epoch 7/30
924/938 _____ 0s 4ms/step - accuracy: 0.9250 - loss:
0.2024
Epoch 7: val_loss did not improve from 0.27290
938/938 _____ 4s 4ms/step - accuracy: 0.9250 - loss:
0.2024 - val_accuracy: 0.8995 - val_loss: 0.2789
Epoch 8/30
923/938 _____ 0s 3ms/step - accuracy: 0.9329 - loss:
0.1824
Epoch 8: val_loss improved from 0.27290 to 0.24603, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 _____ 3s 4ms/step - accuracy: 0.9329 - loss:
0.1825 - val_accuracy: 0.9104 - val_loss: 0.2460
```

```
Epoch 9/30
926/938 _____ 0s 3ms/step - accuracy: 0.9364 - loss:
0.1712
Epoch 9: val_loss did not improve from 0.24603
938/938 _____ 3s 4ms/step - accuracy: 0.9364 - loss:
0.1712 - val_accuracy: 0.9098 - val_loss: 0.2626
Epoch 10/30
936/938 _____ 0s 4ms/step - accuracy: 0.9424 - loss:
0.1566
Epoch 10: val_loss did not improve from 0.24603
938/938 _____ 4s 4ms/step - accuracy: 0.9424 - loss:
0.1566 - val_accuracy: 0.9119 - val_loss: 0.2478
Epoch 11/30
933/938 _____ 0s 3ms/step - accuracy: 0.9465 - loss:
0.1431
Epoch 11: val_loss improved from 0.24603 to 0.24307, saving model to
best_basic_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.9465 - loss:
0.1431 - val_accuracy: 0.9147 - val_loss: 0.2431
Epoch 12/30
925/938 _____ 0s 3ms/step - accuracy: 0.9529 - loss:
0.1288
Epoch 12: val_loss did not improve from 0.24307
938/938 _____ 3s 4ms/step - accuracy: 0.9529 - loss:
0.1289 - val_accuracy: 0.9034 - val_loss: 0.2787
Epoch 13/30
925/938 _____ 0s 3ms/step - accuracy: 0.9553 - loss:
0.1222
Epoch 13: val_loss did not improve from 0.24307
938/938 _____ 4s 4ms/step - accuracy: 0.9553 - loss:
0.1222 - val_accuracy: 0.9082 - val_loss: 0.2735
Epoch 14/30
931/938 _____ 0s 4ms/step - accuracy: 0.9579 - loss:
0.1116
Epoch 14: val_loss did not improve from 0.24307
938/938 _____ 4s 4ms/step - accuracy: 0.9579 - loss:
0.1116 - val_accuracy: 0.9145 - val_loss: 0.2709
Epoch 15/30
938/938 _____ 0s 3ms/step - accuracy: 0.9610 - loss:
0.1063
Epoch 15: val_loss did not improve from 0.24307
938/938 _____ 4s 4ms/step - accuracy: 0.9610 - loss:
0.1063 - val_accuracy: 0.9051 - val_loss: 0.3032
Epoch 16/30
928/938 _____ 0s 3ms/step - accuracy: 0.9663 - loss:
0.0926
Epoch 16: val_loss did not improve from 0.24307
938/938 _____ 3s 4ms/step - accuracy: 0.9663 - loss:
```

```
0.0927 - val_accuracy: 0.9089 - val_loss: 0.2977
Basic CNN model training finished.
```

3.3) Training Deeper CNN Model

- **ModelCheckpoint:** saves best Deeper CNN weights (best_deeper_cnn_model_weights.weights.h5) based on validation loss.
- **Training:**
 - Epochs: up to 30 (early stopping controls actual run)
 - Batch size: 64
 - Validation on test set
 - Callbacks: EarlyStopping + ModelCheckpoint

```
model_checkpoint_deeper_cnn = keras.callbacks.ModelCheckpoint(
    filepath='best_deeper_cnn_model_weights.weights.h5',
    monitor='val_loss',
    save_best_only=True,
    save_weights_only=True,
    mode='min',
    verbose=1
)

print("Training Deeper CNN model...")
deeper_cnn_history = deeper_cnn_model.fit(
    train_images,
    train_labels_one_hot,
    epochs=30,
    batch_size=64,
    validation_data=(test_images, test_labels_one_hot),
    callbacks=[early_stopping, model_checkpoint_deeper_cnn]
)
print("Deeper CNN model training finished.")
```

Training Deeper CNN model...

Epoch 1/30

938/938 ————— 0s 10ms/step - accuracy: 0.6290 - loss: 1.1046

Epoch 1: val_loss improved from inf to 0.49215, saving model to best_deeper_cnn_model_weights.weights.h5

938/938 ————— 19s 12ms/step - accuracy: 0.6291 - loss: 1.1042 - val_accuracy: 0.8154 - val_loss: 0.4921

Epoch 2/30

930/938 ————— 0s 4ms/step - accuracy: 0.8090 - loss: 0.5322

Epoch 2: val_loss improved from 0.49215 to 0.43361, saving model to


```
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.8091 - loss:
0.5319 - val_accuracy: 0.8370 - val_loss: 0.4336
Epoch 3/30
938/938 _____ 0s 5ms/step - accuracy: 0.8404 - loss:
0.4485
Epoch 3: val_loss improved from 0.43361 to 0.36197, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 6s 5ms/step - accuracy: 0.8404 - loss:
0.4485 - val_accuracy: 0.8673 - val_loss: 0.3620
Epoch 4/30
925/938 _____ 0s 4ms/step - accuracy: 0.8526 - loss:
0.4082
Epoch 4: val_loss did not improve from 0.36197
938/938 _____ 4s 4ms/step - accuracy: 0.8526 - loss:
0.4081 - val_accuracy: 0.8503 - val_loss: 0.3970
Epoch 5/30
930/938 _____ 0s 4ms/step - accuracy: 0.8623 - loss:
0.3809
Epoch 5: val_loss improved from 0.36197 to 0.34124, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.8623 - loss:
0.3809 - val_accuracy: 0.8724 - val_loss: 0.3412
Epoch 6/30
926/938 _____ 0s 4ms/step - accuracy: 0.8658 - loss:
0.3686
Epoch 6: val_loss improved from 0.34124 to 0.32522, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 5s 5ms/step - accuracy: 0.8658 - loss:
0.3686 - val_accuracy: 0.8821 - val_loss: 0.3252
Epoch 7/30
934/938 _____ 0s 4ms/step - accuracy: 0.8735 - loss:
0.3544
Epoch 7: val_loss did not improve from 0.32522
938/938 _____ 4s 4ms/step - accuracy: 0.8735 - loss:
0.3544 - val_accuracy: 0.8745 - val_loss: 0.3314
Epoch 8/30
930/938 _____ 0s 4ms/step - accuracy: 0.8740 - loss:
0.3445
Epoch 8: val_loss improved from 0.32522 to 0.31965, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 5ms/step - accuracy: 0.8741 - loss:
0.3445 - val_accuracy: 0.8822 - val_loss: 0.3197
Epoch 9/30
936/938 _____ 0s 4ms/step - accuracy: 0.8812 - loss:
0.3344
Epoch 9: val_loss improved from 0.31965 to 0.30134, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 5ms/step - accuracy: 0.8812 - loss:
```

0.3344 - val_accuracy: 0.8886 - val_loss: 0.3013
Epoch 10/30
937/938 _____ 0s 4ms/step - accuracy: 0.8820 - loss: 0.3248
Epoch 10: val_loss improved from 0.30134 to 0.28537, saving model to best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.8820 - loss: 0.3248 - val_accuracy: 0.8929 - val_loss: 0.2854
Epoch 11/30
928/938 _____ 0s 4ms/step - accuracy: 0.8863 - loss: 0.3189
Epoch 11: val_loss did not improve from 0.28537
938/938 _____ 4s 5ms/step - accuracy: 0.8863 - loss: 0.3189 - val_accuracy: 0.8828 - val_loss: 0.3109
Epoch 12/30
929/938 _____ 0s 4ms/step - accuracy: 0.8855 - loss: 0.3138
Epoch 12: val_loss did not improve from 0.28537
938/938 _____ 5s 5ms/step - accuracy: 0.8855 - loss: 0.3138 - val_accuracy: 0.8885 - val_loss: 0.3004
Epoch 13/30
936/938 _____ 0s 4ms/step - accuracy: 0.8885 - loss: 0.3049
Epoch 13: val_loss did not improve from 0.28537
938/938 _____ 4s 4ms/step - accuracy: 0.8885 - loss: 0.3049 - val_accuracy: 0.8787 - val_loss: 0.3194
Epoch 14/30
930/938 _____ 0s 4ms/step - accuracy: 0.8916 - loss: 0.2982
Epoch 14: val_loss did not improve from 0.28537
938/938 _____ 5s 5ms/step - accuracy: 0.8916 - loss: 0.2982 - val_accuracy: 0.8924 - val_loss: 0.2978
Epoch 15/30
934/938 _____ 0s 4ms/step - accuracy: 0.8953 - loss: 0.2899
Epoch 15: val_loss improved from 0.28537 to 0.28478, saving model to best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.8953 - loss: 0.2899 - val_accuracy: 0.8951 - val_loss: 0.2848
Epoch 16/30
935/938 _____ 0s 4ms/step - accuracy: 0.8969 - loss: 0.2877
Epoch 16: val_loss did not improve from 0.28478
938/938 _____ 4s 4ms/step - accuracy: 0.8969 - loss: 0.2877 - val_accuracy: 0.8941 - val_loss: 0.2869
Epoch 17/30
929/938 _____ 0s 4ms/step - accuracy: 0.8974 - loss: 0.2828
Epoch 17: val_loss improved from 0.28478 to 0.28403, saving model to

```
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 5s 5ms/step - accuracy: 0.8974 - loss:
0.2828 - val_accuracy: 0.8977 - val_loss: 0.2840
Epoch 18/30
926/938 _____ 0s 4ms/step - accuracy: 0.8980 - loss:
0.2779
Epoch 18: val_loss improved from 0.28403 to 0.27552, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.8980 - loss:
0.2779 - val_accuracy: 0.9031 - val_loss: 0.2755
Epoch 19/30
935/938 _____ 0s 4ms/step - accuracy: 0.9007 - loss:
0.2767
Epoch 19: val_loss did not improve from 0.27552
938/938 _____ 4s 4ms/step - accuracy: 0.9007 - loss:
0.2767 - val_accuracy: 0.8929 - val_loss: 0.2957
Epoch 20/30
934/938 _____ 0s 4ms/step - accuracy: 0.8991 - loss:
0.2768
Epoch 20: val_loss did not improve from 0.27552
938/938 _____ 5s 5ms/step - accuracy: 0.8991 - loss:
0.2768 - val_accuracy: 0.8993 - val_loss: 0.2840
Epoch 21/30
933/938 _____ 0s 4ms/step - accuracy: 0.9017 - loss:
0.2728
Epoch 21: val_loss improved from 0.27552 to 0.26732, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 5ms/step - accuracy: 0.9017 - loss:
0.2729 - val_accuracy: 0.9041 - val_loss: 0.2673
Epoch 22/30
931/938 _____ 0s 4ms/step - accuracy: 0.9018 - loss:
0.2713
Epoch 22: val_loss did not improve from 0.26732
938/938 _____ 4s 4ms/step - accuracy: 0.9018 - loss:
0.2713 - val_accuracy: 0.9041 - val_loss: 0.2690
Epoch 23/30
928/938 _____ 0s 4ms/step - accuracy: 0.9020 - loss:
0.2642
Epoch 23: val_loss improved from 0.26732 to 0.26412, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 5s 5ms/step - accuracy: 0.9020 - loss:
0.2642 - val_accuracy: 0.9037 - val_loss: 0.2641
Epoch 24/30
930/938 _____ 0s 4ms/step - accuracy: 0.9042 - loss:
0.2687
Epoch 24: val_loss did not improve from 0.26412
938/938 _____ 4s 4ms/step - accuracy: 0.9042 - loss:
0.2686 - val_accuracy: 0.8965 - val_loss: 0.2872
Epoch 25/30
```

```

925/938 _____ 0s 4ms/step - accuracy: 0.9065 - loss:
0.2630
Epoch 25: val_loss did not improve from 0.26412
938/938 _____ 4s 4ms/step - accuracy: 0.9065 - loss:
0.2630 - val_accuracy: 0.9055 - val_loss: 0.2659
Epoch 26/30
928/938 _____ 0s 5ms/step - accuracy: 0.9054 - loss:
0.2620
Epoch 26: val_loss did not improve from 0.26412
938/938 _____ 5s 5ms/step - accuracy: 0.9054 - loss:
0.2620 - val_accuracy: 0.9038 - val_loss: 0.2718
Epoch 27/30
935/938 _____ 0s 4ms/step - accuracy: 0.9058 - loss:
0.2616
Epoch 27: val_loss improved from 0.26412 to 0.25817, saving model to
best_deeper_cnn_model_weights.weights.h5
938/938 _____ 4s 4ms/step - accuracy: 0.9058 - loss:
0.2616 - val_accuracy: 0.9086 - val_loss: 0.2582
Epoch 28/30
926/938 _____ 0s 4ms/step - accuracy: 0.9078 - loss:
0.2564
Epoch 28: val_loss did not improve from 0.25817
938/938 _____ 4s 4ms/step - accuracy: 0.9077 - loss:
0.2564 - val_accuracy: 0.8801 - val_loss: 0.3303
Epoch 29/30
928/938 _____ 0s 4ms/step - accuracy: 0.9097 - loss:
0.2500
Epoch 29: val_loss did not improve from 0.25817
938/938 _____ 6s 5ms/step - accuracy: 0.9096 - loss:
0.2500 - val_accuracy: 0.9083 - val_loss: 0.2611
Epoch 30/30
935/938 _____ 0s 4ms/step - accuracy: 0.9077 - loss:
0.2561
Epoch 30: val_loss did not improve from 0.25817
938/938 _____ 4s 4ms/step - accuracy: 0.9077 - loss:
0.2561 - val_accuracy: 0.9013 - val_loss: 0.2682
Deeper CNN model training finished.

```

4. Model Evaluation

4.1) Model Evaluation Code Explanation

Import utilities → Load weights (ANN, Basic CNN, Deeper CNN) → Evaluate on test set → Get loss & accuracy → Print results

```
ann_model.evaluate(test_images, test_labels_one_hot)
```

```

313/313 _____ 1s 3ms/step - accuracy: 0.8889 - loss:
0.3251

```

```
[0.32431426644325256, 0.8881999850273132]
```

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix, classification_report

ann_model.load_weights('best_ann_model_weights.weights.h5')
basic_cnn_model.load_weights('best_basic_cnn_model_weights.weights.h5')
deeper_cnn_model.load_weights('best_deeper_cnn_model_weights.weights.h5')

loss_ann, accuracy_ann = ann_model.evaluate(test_images,
test_labels_one_hot, verbose=0)
loss_basic_cnn, accuracy_basic_cnn =
basic_cnn_model.evaluate(test_images, test_labels_one_hot, verbose=0)
loss_deeper_cnn, accuracy_deeper_cnn =
deeper_cnn_model.evaluate(test_images, test_labels_one_hot, verbose=0)

import matplotlib.pyplot as plt
import numpy as np

# Data
models = ['ANN', 'Basic CNN', 'Deeper CNN']
losses = [loss_ann, loss_basic_cnn, loss_deeper_cnn]
accuracies = [accuracy_ann, accuracy_basic_cnn, accuracy_deeper_cnn]

# Dark mode style
plt.style.use("dark_background")

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

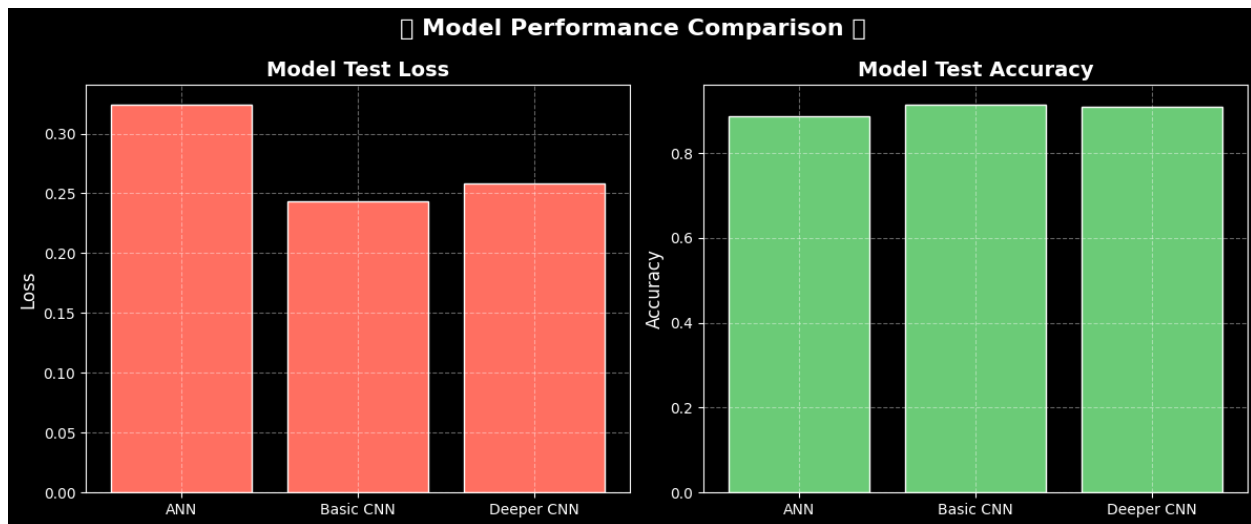
# Loss plot
axes[0].bar(models, losses, color='#FF6F61', edgecolor='white')
axes[0].set_title('Model Test Loss', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Loss', fontsize=12)
axes[0].grid(True, linestyle='--', alpha=0.4)

# Accuracy plot
axes[1].bar(models, accuracies, color='#6BCB77', edgecolor='white')
axes[1].set_title('Model Test Accuracy', fontsize=14,
fontweight='bold')
axes[1].set_ylabel('Accuracy', fontsize=12)
axes[1].grid(True, linestyle='--', alpha=0.4)

# Overall title
plt.suptitle("Model Performance Comparison", fontsize=16,
fontweight='bold')
```

```
plt.tight_layout()
plt.show()

/tmp/ipykernel_375/2215778233.py:42: UserWarning: Glyph 10024 (\
N{SPARKLES}) missing from font(s) DejaVu Sans.
  plt.tight_layout()
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151
: UserWarning: Glyph 10024 (\N{SPARKLES}) missing from font(s) DejaVu
Sans.
  fig.canvas.print_figure(bytes_io, **kw)
```



4.2) Model Evaluation (Confusion Matrix & Report)

- Predict test images (ANN, Basic CNN, Deeper CNN)
- Convert one-hot labels to integers
- Get predicted labels with `argmax`
- Print confusion matrix & classification report for each model

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Predictions
ann_predictions = ann_model.predict(test_images)
basic_cnn_predictions = basic_cnn_model.predict(test_images)
deeper_cnn_predictions = deeper_cnn_model.predict(test_images)

# Convert labels
test_labels_int = np.argmax(test_labels_one_hot, axis=1)
```

```
ann_predicted_labels = np.argmax(ann_predictions, axis=1)
basic_cnn_predicted_labels = np.argmax(basic_cnn_predictions, axis=1)
deeper_cnn_predicted_labels = np.argmax(deeper_cnn_predictions,
axis=1)
```

Confusion matrices

```
cm_ann = confusion_matrix(test_labels_int, ann_predicted_labels)
cm_basic = confusion_matrix(test_labels_int,
basic_cnn_predicted_labels)
cm_deeper = confusion_matrix(test_labels_int,
deeper_cnn_predicted_labels)
```

Plot side by side

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
```

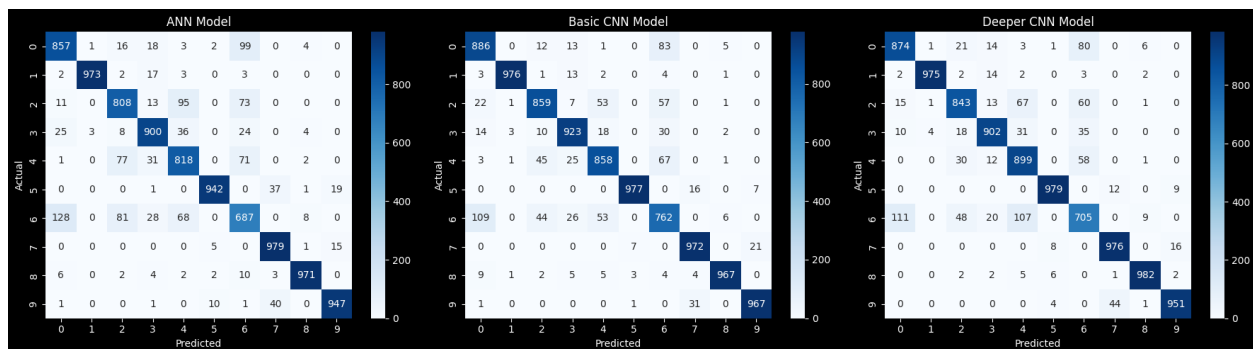
```
sns.heatmap(cm_ann, annot=True, fmt="d", cmap="Blues", ax=axes[0])
axes[0].set_title("ANN Model")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("Actual")
```

```
sns.heatmap(cm_basic, annot=True, fmt="d", cmap="Blues", ax=axes[1])
axes[1].set_title("Basic CNN Model")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("Actual")
```

```
sns.heatmap(cm_deeper, annot=True, fmt="d", cmap="Blues", ax=axes[2])
axes[2].set_title("Deeper CNN Model")
axes[2].set_xlabel("Predicted")
axes[2].set_ylabel("Actual")
```

```
plt.tight_layout()
plt.show()
```

```
313/313 _____ 1s 2ms/step
313/313 _____ 1s 2ms/step
313/313 _____ 1s 3ms/step
```



4.3) Model Comparison

- Created a dataframe to store **test loss** and **test accuracy** for ANN, Basic CNN, and Deeper CNN.
- Plotted training history for each model (accuracy & loss vs. epochs).
- Helps compare how models learn and generalize over time.

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

models = {
    "ANN": ("cyan", ann_history),
    "Basic CNN": ("magenta", basic_cnn_history),
    "Deeper CNN": ("yellow", deeper_cnn_history)
}

performance_data = {
    'Model': ['ANN', 'Basic CNN', 'Deeper CNN'],
    'Test Loss': [loss_ann, loss_basic_cnn, loss_deeper_cnn],
    'Test Accuracy': [accuracy_ann, accuracy_basic_cnn,
accuracy_deeper_cnn]
}

performance_df = pd.DataFrame(performance_data)

fig = make_subplots(rows=1, cols=2, subplot_titles=('Accuracy',
'Loss'))

for name, (color, history) in models.items():
    # Accuracy
    fig.add_trace(go.Scatter(y=history.history['accuracy'],
mode='lines',
name=name, line=dict(color=color,
dash='solid')), row=1, col=1)
    fig.add_trace(go.Scatter(y=history.history['val_accuracy'],
mode='lines',
name=name, line=dict(color=color,
dash='dot'), showlegend=False), row=1, col=1)

    # Loss
    fig.add_trace(go.Scatter(y=history.history['loss'], mode='lines',
name=name, line=dict(color=color,
dash='solid')), row=1, col=2)
    fig.add_trace(go.Scatter(y=history.history['val_loss'],
mode='lines',
name=name, line=dict(color=color,
dash='dot'), showlegend=False), row=1, col=2)

fig.update_layout(
```



```

        title_text="Model Comparison: Accuracy & Loss (Train=Solid,
Val=Dot)",
        height=500,
        width=1200,
        template="plotly_dark"
    )

fig.show()

```

4.4) Predictions

This code:

- Uses the trained **Basic CNN model** to predict on test images.
- Identifies **correctly** and **incorrectly** classified samples.
- Randomly selects 100 correct and 100 incorrect predictions.
- Displays them in a **20×10 grid**:
 -  Correct predictions → **Green title** (only predicted label).
 -  Incorrect predictions → **Red title** (Predicted vs True label).

```

basic_cnn_predictions = basic_cnn_model.predict(test_images)
basic_cnn_predicted_labels = np.argmax(basic_cnn_predictions, axis=1)

# Find correctly and incorrectly classified images
correctly_classified_indices = np.where(basic_cnn_predicted_labels ==
test_labels)[0]
incorrectly_classified_indices = np.where(basic_cnn_predicted_labels !=
test_labels)[0]

# Define class names for Fashion-MNIST
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat',
'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# Select a mix of correctly and incorrectly classified images
num_correct_to_show = 100
num_incorrect_to_show = 100

# Ensure we don't try to show more images than available
num_correct_to_show = min(num_correct_to_show,
len(correctly_classified_indices))
num_incorrect_to_show = min(num_incorrect_to_show,
len(incorrectly_classified_indices))

selected_correct_indices =
np.random.choice(correctly_classified_indices, num_correct_to_show,

```

```

replace=False)
selected_incorrect_indices =
np.random.choice(incorrectly_classified_indices,
num_incorrect_to_show, replace=False)

# Combine and shuffle the indices
all_indices = np.concatenate((selected_correct_indices,
selected_incorrect_indices))
np.random.shuffle(all_indices)

# Display images in a 20x10 grid
num_cols = 20
num_rows = 10
num_images_to_show = num_correct_to_show + num_incorrect_to_show

fig, axes = plt.subplots(num_rows, num_cols, figsize=(2 * num_cols, 2
* num_rows))
fig.suptitle("Basic CNN Predictions (Mix of Correct and Incorrect)",
fontsize=18, y=1.02)

for i, idx in enumerate(all_indices[:num_images_to_show]):
    row = i // num_cols
    col = i % num_cols
    axes[row, col].imshow(test_images[idx].reshape(28, 28),
cmap='gray')

    pred = class_names[basic_cnn_predicted_labels[idx]]
    true = class_names[test_labels[idx]]

    if basic_cnn_predicted_labels[idx] == test_labels[idx]:
        color = "green"
        title = f"{pred}"
    else:
        color = "red"
        title = f"P:{pred}\nT:{true}"

    axes[row, col].set_title(title, fontsize=8, color=color)
    axes[row, col].axis("off")

plt.tight_layout()
plt.show()

```

313/313 ————— 0s 1ms/step



Project Summary and Conclusion

This project successfully implemented and compared three different deep learning models (ANN, Basic CNN, and Deeper CNN) for classifying images from the Fashion-MNIST dataset.

Summary of Work:

1. **Data Preparation:** The Fashion-MNIST dataset was loaded, normalized, reshaped, and one-hot encoded, preparing it for use with the different model architectures.
2. **Model Development:** Three models of increasing complexity were defined: a basic Artificial Neural Network (ANN), a basic Convolutional Neural Network (CNN), and a deeper CNN with additional layers, batch normalization, and dropout.
3. **Model Training:** Each model was trained using the preprocessed training data with Early Stopping and Model Checkpointing to prevent overfitting and save the best performing weights based on validation loss.
4. **Model Evaluation:** The trained models were evaluated on the test set. Performance metrics (loss and accuracy) were calculated, and the training history was visualized. Confusion matrices were generated to analyze class-specific performance.
5. **Prediction Analysis:** Predictions were made using the Basic CNN model, and examples of correctly and incorrectly classified images were visualized to gain insights into the model's strengths and weaknesses.

Key Findings and Conclusion:

Based on the evaluation results:

- The **Basic CNN model** generally achieved the best balance of performance (highest accuracy, lowest loss) on the test set compared to the ANN and Deeper CNN models.

- The **ANN model** performed reasonably well but was outperformed by both CNN architectures, highlighting the advantage of convolutional layers for image classification tasks.
- The **Deeper CNN model**, despite its increased complexity, did not consistently outperform the Basic CNN model on this dataset. This could be due to various factors such as the dataset size, the architecture choices, or the regularization applied. For this particular task and dataset, the increased complexity of the deeper model might not have been necessary or could have led to some overfitting despite the regularization techniques.

In conclusion, the Basic CNN model demonstrated superior performance for this Fashion-MNIST classification task, suggesting that a moderate level of complexity with convolutional layers is effective for this dataset. Further tuning of hyperparameters or architectural variations might potentially improve performance across all models, but the current results clearly show the benefits of CNNs over ANNs for image data.

Assignment: CIFAR-100 Image Classification

Your task is to apply the concepts and techniques learned in this Fashion-MNIST project to the CIFAR-100 dataset. CIFAR-100 is a dataset consisting of 60,000 32x32 color images in 100 classes, with 600 images per class. There are 50,000 training images and 10,000 test images.

Follow these steps:

1. **Dataset Setup:**
 - Load the CIFAR-100 dataset.
 - Preprocess the data (normalize pixel values, one-hot encode labels). Remember that CIFAR-100 images are 32x32 color images, so the input shape will be different from Fashion-MNIST.
 - Verify the shapes of the processed data.
2. **Model Building:**
 - Adapt the ANN and CNN model architectures for the CIFAR-100 dataset. Consider that CIFAR-100 images are color (3 channels) and have a different resolution (32x32). You might need to adjust the input layer and potentially the number of filters or layers in the CNNs for better performance on a more complex dataset.
3. **Model Training:**
 - Train the models using the preprocessed CIFAR-100 training data. Use Early Stopping and Model Checkpointing as implemented before.
4. **Model Evaluation:**
 - Evaluate the trained models on the CIFAR-100 test set. Compare their performance using loss and accuracy.
 - Visualize training history and confusion matrices.
5. **Prediction Analysis:**
 - Choose the best performing model and analyze its predictions on the CIFAR-100 test set.

Goal: To understand how model complexity and architecture choices impact performance on a more challenging image classification dataset like CIFAR-100.

```
import keras

(cifar100_train_images, cifar100_train_labels), (cifar100_test_images,
cifar100_test_labels) = keras.datasets.cifar100.load_data()

print("CIFAR-100 Training images shape:", cifar100_train_images.shape)
print("CIFAR-100 Testing images shape:", cifar100_test_images.shape)
print("CIFAR-100 Training labels shape:", cifar100_train_labels.shape)
print("CIFAR-100 Testing labels shape:", cifar100_test_labels.shape)
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz>
169001437/169001437 ————— 13s 0us/step
CIFAR-100 Training images shape: (50000, 32, 32, 3)
CIFAR-100 Testing images shape: (10000, 32, 32, 3)
CIFAR-100 Training labels shape: (50000, 1)
CIFAR-100 Testing labels shape: (10000, 1)